

IIoT2026 Review: High-Performance Telemetry Visualization: Native C++ Architectures vs. Standard Web-Stack Baselines in Factory Workstations

1 Overview

This paper investigates the architectural trade-off between native C++/GTK4 and web-based solutions for high-frequency industrial telemetry visualization. The authors compare several implementations, including Flask with DOM rendering, Flask with HTML5 Canvas, Node.js with Canvas, and a native C++ implementation. The evaluation considers a workload of 5,000 MQTT messages per second and reports differences in memory usage, CPU consumption, and rendering performance.

The topic is relevant to industrial IoT, edge computing, and resource-efficient monitoring systems. The comparison between native and web-based visualization approaches addresses a practical problem faced by industrial edge platforms. The inclusion of multiple web baselines and a native implementation is also a positive aspect.

However, the main conclusions of the paper are not sufficiently supported by the experimental methodology. Several fundamental issues affect the validity of the reported comparisons. The study does not demonstrate that all architectures process an equivalent effective workload, CPU measurements are not performed with the same scope, browser memory usage is estimated using metrics that are problematic for a multi-process Chromium architecture, and important metrics such as end-to-end telemetry latency are missing.

Furthermore, some conclusions are significantly stronger than the presented evidence. In particular, the claim regarding fundamental limitations of Python asynchronous ecosystems is based on a limited extended-run observation under a specific configuration rather than a comprehensive experimental analysis.

Although the paper addresses an important problem, these issues affect the scientific validity of the main conclusions. Therefore, the recommendation is **Reject**.

2 Strengths

1. The paper addresses an important and practical problem in industrial IoT systems, where visualization software must operate under resource constraints while processing high-rate telemetry streams.

2. The experimental comparison is broader than a simple native-versus-web comparison. The authors include both inefficient and optimized web approaches, which provides useful engineering observations regarding DOM-based rendering, Canvas rendering, and backend architecture choices.
3. The manuscript provides source code and benchmark artifacts, which is a positive step toward reproducibility.
4. The authors acknowledge some limitations of their measurements, including the impact of the WSL2 environment and differences in CPU measurement scope.

3 Major Technical Concerns

1. **The effective workload processed by each architecture is not verified.**

The evaluation uses MQTT QoS 0, but the paper does not verify end-to-end message delivery at the subscriber level. Therefore, it is unclear whether the native, Flask, and Node.js implementations actually received and processed the same number of messages.

This is a fundamental issue because an implementation that drops messages or fails to keep up with the publisher may appear more efficient in terms of CPU, memory, or responsiveness. The authors should report message counts at the publisher, broker, backend/subscriber, and visualization stages.

2. **CPU measurements are not comparable across architectures.**

The native C++ CPU measurement includes the complete application, including rendering and synchronization overhead. In contrast, the reported Node.js and Flask CPU values mainly represent backend processing and exclude the Chromium renderer.

Consequently, the paper does not provide a fair comparison of total system CPU consumption. All architectures should be evaluated using the same measurement scope, including the complete process tree.

3. **The memory comparison methodology is insufficient.**

The paper uses RSS/RES-based measurements to compare memory usage. However, Chromium is a multi-process application with shared libraries and memory pages. Summing RSS values from multiple browser processes can count shared memory more than once and may not represent actual physical memory consumption.

More appropriate measurements such as PSS, USS, or system-level memory accounting should be reported before making strong claims regarding memory advantages.

4. **Latency is discussed but not measured.**

The paper presents architectural arguments related to responsiveness and latency, but it does not report end-to-end telemetry latency from message publication to visible display.

Frame rate measurements alone are insufficient because a system may maintain a high frame rate while displaying stale data. Publish-to-display latency and tail latency statistics should be provided.

5. The FPS and Vsync claims require stronger evidence.

The reported 60 FPS result appears to be based on update timing rather than direct measurement of presented frames. In my opinion, a timer frequency of approximately 60 Hz does not prove actual frame presentation rate or synchronization with display refresh.

The authors should provide measurements from the rendering framework itself.

6. The Flask/eventlet conclusion is overstated.

The reported memory growth is based on an informal extended experiment rather than the main benchmark protocol. Moreover, the observation is used to make broad claims about Python asynchronous ecosystems.

A single configuration-specific observation is insufficient to establish a general limitation. In my opinion, additional experiments with properly configured eventlet, threading, gevent, and asyncio-based implementations would be required.

7. The evaluation scope is limited.

The study evaluates one primary workload configuration. The conclusions may change under different message rates, payload sizes, QoS settings, burst patterns, multiple telemetry streams, longer execution periods, or different operating systems and rendering environments.

8. The related-work base is too limited.

Only three references are provided. The paper needs stronger scholarly positioning with prior work on MQTT benchmarking, native-versus-web desktop performance, browser process/memory accounting, GUI rendering and frame pacing, embedded web runtimes, and asynchronous concurrency frameworks.

4 Key Questions

1. What percentage of published MQTT messages are successfully delivered and processed by each architecture at the publisher, broker, backend, and display boundaries?
2. How would the CPU comparison change if the complete application stack, including Chromium rendering, were included for all web-based architectures?
3. How was browser memory calculated? What are the corresponding PSS/USS values for the complete Chromium process tree?
4. What are the actual end-to-end telemetry latency values (median, p95, and p99) from publication to visible display?
5. Does the reported Flask/eventlet memory growth reproduce under properly configured environments and alternative asynchronous frameworks?
6. How robust are the conclusions when workload parameters such as message rate, payload size, QoS level, and execution duration are changed?